



Ejercicios - Semana 14



TEMA 1

CREACIÓN DE JOBS Y ALERTS CON SQL SERVER AGENT

★ ¡MARIO DBA AL RESCATE DE TU BASE DE DATOS! ★



MISIÓN PRINCIPAL

Automatizar tareas y monitorear eventos críticos en SQL Server para mantener la base de datos siempre saludable.



1 ENUNCIADO

La Oficina de Grados y Títulos requiere que todos los días a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.

Se debe crear un Job en SQL Server Agent, programarlo diariamente, y registrar el resultado en una tabla de auditoría.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
-- Contar trámites del día anterior
SELECT COUNT(*) AS TotalTramites
FROM Trámite
WHERE FechaRegistro = CAST(DATEADD(D,-1, GETDATE()) AS DATE);

-- Registrar resultado
INSERT INTO ReporteDiario
(NombreJob, FechaJob, Mensaje)
VALUES ('ReporteDiario - EXITO',
GETDATE(), 'Reporte generado correctamente.');
```



SQL Server Agent
Job: ReporteDiario
Schedule: Diario
Hora: 08:00 AM



3 JUSTIFICACIÓN TÉCNICA

- El conteo se realiza sobre el día anterior usando GETDATE().
- Se registra el resultado en la tabla ReporteDiario para trazabilidad.
- El Job se programa diariamente a las 08:00 AM con SQL Server Agent.

4 BUENAS PRÁCTICAS

- Separación de responsabilidades.
- Registro estructurado de resultados.
- Manejo de errores (TRY...CATCH).
- Reintentos configurados en el Job.



ALERTS: TU SISTEMA DE DEFENSA

- 🔔 Crea un Alert para errores de severidad 16-25.
- ✉ Envía notificación por correo al DBA.
- ★ Asegura respuesta rápida ante fallos del Job.





MISIÓN PRINCIPAL:

¡Ayuda a Mario a mantener actualizados los datos de los trámites en proceso y deja huella de cada ejecución para proteger el historial del Reino Champiñón!



1 ENUNCIADO DEL PROBLEMA

Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado EN_PROCESO, registrando la ejecución.

REQUERIMIENTOS:

- Actualizar FechaUltMod de los trámites en estado EN_PROCESO.
- Usar la sentencia UPDATE.
- Configurar un Job en SQL Server Agent para ejecución semanal.
- Registrar el resultado de cada ejecución en una tabla histórica.

ENTIDADES INVOLUCRADAS



¿QUÉ HACE ESTE PROYECTO?



2 SCRIPT T-SQL (SOLUCIÓN COMPLETA)

```

-- 1. CREACIÓN DE LA TABLA HISTÓRICA / AUDITORÍA DE ACTUALIZACIONES
USE GradosTitulos;
GO

IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID('dbo.HistorialModificaciones'))
BEGIN
    CREATE TABLE dbo.HistorialModificaciones (
        HistoriaID INT IDENTITY(1,1) NOT NULL,
        FechaEjecucion DATETIME DEFAULT GETDATE(),
        CantidadRegistrosActualizados INT NOT NULL,
        EstadoAfectado VARCHAR(20) NOT NULL,
        CONSTRAINT PK_HistorialModificaciones PRIMARY KEY CLUSTERED (HistoriaID)
    );
END;
GO

-- 2. CONFIGURACIÓN DEL JOB EN SQL SERVER AGENT
USE msdb;
GO

DECLARE @jobName NVARCHAR(128) = N'Job_Actualizacion_Semanal_Tramites';
DECLARE @jobId BINARY(16);

-- Verificación de idempotencia: eliminar si existe
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @jobName)
BEGIN
    EXEC msdb.dbo.sp_delete_job @job_name = @jobName, @delete_unused_schedule = 1;
END;

-- Crear el Job
EXEC msdb.dbo.sp_add_job
    @job_name = @jobName,
    @enabled = 1,
    @description = N'Actualiza semanalmente FechaUltMod de los trámites EN_PROCESO.',
    @category_name = N'Uncategorized (Local)',
    @owner_login_name = N'sa',
    @job_id = @jobId OUTPUT;
  
```

3 JUSTIFICACIÓN TÉCNICA

- Uso de @@ROWCOUNT:** captura el número exacto de filas modificadas por el UPDATE, permitiendo almacenar métricas reales y precisas en el historial.
- Manejo del Contexto de Transacción Implícita:** el UPDATE y el registro en la tabla histórica ocurren dentro del mismo step T-SQL, garantizando consistencia en la ejecución del Job.
- Filtros por Estado Semántico:** al filtrar por 'EN_PROCESO', se evitan actualizaciones innecesarias en filas estáticas, reduciendo bloqueos y mejorando el rendimiento.
- Historial Operacional:** la tabla HistorialModificaciones permite auditoría de negocio sin depender únicamente del historial interno de msdb.dbo.sysjobhistory.

4 BUENAS PRÁCTICAS

- Aislamiento de la Programación Semanal:** Se configuró @freq_type = 8 (semanal) para los domingos a la medianoche (@active_start_time = 0), ventana óptima que minimiza impacto y contención de recursos.
- Automatización de Historial Interno y Externo:** Además del log interno en msdb.dbo.sysjobhistory, se crea una tabla histórica en la BD de negocio (HistorialModificaciones) para consulta directa por el área de negocio y desarrollo.
- Parámetros de Reintento Elásticos:** Se definieron 2 intentos y 10 minutos de intervalo para asegurar resiliencia ante bloqueos temporales por mantenimientos o respaldos dominicales.
- Seguridad y Control:** El Job se crea bajo el propietario 'sa' y se puede asignar a una categoría específica para control y monitoreo.

5 DIAGRAMA DEL FLUJO DE PROCESO





MISIÓN PRINCIPAL:

¡Ayuda a Mario a limpiar los registros temporales antiguos, guardando su historia y manteniendo nuestro sistema ágil y seguro!



1 ENUNCIADO DEL PROBLEMA

Diseñar un Job que elimine mensualmente los registros temporales de la tabla `Tramite.LogTemporal`. Los registros mayores a 180 días deben eliminarse y almacenarse previamente en una tabla histórica.

REQUERIMIENTOS OBLIGATORIOS:

- INSERT INTO Histórico (Migración de datos).
- DELETE (Depuración de la tabla original).
- Bloques de control de errores TRY...CATCH.
- Gestión explícita de Transacciones.
- Automatización mediante SQL Server Agent.

FLUJO DEL PROCESO



2 SCRIPT T-SQL (SOLUCIÓN COMPLETA)

```

/* 1. CREACIÓN DE LA TABLA HISTÓRICA DE RESPALDO */
USE GradosTitulos;
GO
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID('Tramite.LogTemporal_Historico'))
BEGIN
    CREATE TABLE [Tramite].[LogTemporal_Historico] (
        [LogHistoricoID] INT IDENTITY(1,1) NOT NULL,
        [LogID] INT NOT NULL, -- ID original de la tabla temporal
        [Detalle] NVARCHAR(MAX),
        [FechaRegistro] DATETIME NOT NULL,
        [FechaMigracion] DATETIME DEFAULT GETDATE(),
        CONSTRAINT [PK_LogTemporal_Historico] PRIMARY KEY CLUSTERED ([LogHistoricoID])
    );
END;
GO

/* 2. CONFIGURACIÓN DEL JOB EN SQL SERVER AGENT */
USE msdb;
GO
DECLARE @jobName NVARCHAR(128) = N'Job_Depuración_Mensual_LogTemporal';
DECLARE @jobId BINARY(16);

-- Garantizar idempotencia
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @jobName)
BEGIN
    EXEC msdb.dbo.sp_delete_job @job_name = @jobName, @delete_unused_schedule = 1;
END;

-- Crear el Job
EXEC msdb.dbo.sp_add_job
    @job_name = @jobName,
    @enabled = 1,
    @description = N'Migra a histórico y elimina registros con más de 180 días en Tramite.LogTemporal',
    @category_name = N'[Uncategorized (Local)]',
    @owner_login_name = N'sa',
    @job_id = @jobId OUTPUT;
  
```

3 JUSTIFICACIÓN TÉCNICA



Transacciones Explícitas:

El proceso consiste en INSERT seguido de DELETE. La transacción asegura atomicidad: ambos comandos se completan o ninguno se ejecuta.



Estructura TRY...CATCH:

Captura errores en tiempo de ejecución (espacio en disco, bloqueos, etc.) y garantiza la liberación de recursos mediante ROLLBACK.



Uso de RAISERROR:

Al capturar un error, se relanza para que SQL Server Agent registre el Job como fallido y no lo asuma exitoso de forma silenciosa.



Rendimiento y Consistencia:

El uso de transacciones y filtros por antigüedad reduce bloqueos innecesarios y mantiene la integridad de los datos.

4 BUENAS PRÁCTICAS DEL DBA



Cálculo dinámico con DATEADD:

Se utiliza `DATEADD(DAY, -180, GETDATE())` para definir dinámicamente la frontera de los 180 días en cada ejecución.



Programación en horas de baja carga:

@freq_type = 16 (mensual) el primer día del mes a la 01:00 AM (@active_start_time = 10000). Evita impacto en horas de oficina.



SET NOCOUNT ON:

Reduce el tráfico interno de mensajes ("X filas afectadas"), optimizando la ejecución del script dentro del subsistema del Agente.



Auditoría y Trazabilidad:

Se crea una tabla histórica con `FechaMigracion` para consulta de auditoría y recuperación de información.



Idempotencia del Job:

El script elimina el Job si ya existe antes de crearlo nuevamente, evitando duplicados.



5 SCRIPT DEL STEP (T-SQL) - MIGRACIÓN Y DEPURACIÓN

```

-- Step: Migrar a histórico y eliminar registros (> 180 días)
EXEC msdb.dbo.sp_add_jobstep
    @job_id = @jobId,
    @step_id = 1,
    @step_name = N'Migrar y depurar LogTemporal',
    @subsystem = N'TSQL',
    @database_name = N'GradosTitulos',
    @command = N'SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @FechaLimite DATETIME = DATEADD(DAY, -180, GETDATE());
        -- 1. Migrar registros antiguos a la tabla histórica
        INSERT INTO [Tramite].[LogTemporal_Historico] (LogID, Detalle, FechaRegistro, FechaMigracion)
        SELECT LogID, Detalle, FechaRegistro, GETDATE()
        FROM [Tramite].[LogTemporal]
        WHERE FechaRegistro < @FechaLimite;
    
```

```

    -- 2. Eliminar registros migrados
    DELETE FROM [Tramite].[LogTemporal]
    WHERE FechaRegistro < @FechaLimite;

    COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        DECLARE @ErrorMensaje NVARCHAR(4000) = ERROR_MESSAGE();
        RAISERROR('Error al migrar o eliminar registros en LogTemporal: %s',
            16, 1, @ErrorMensaje);
    END CATCH;
    @retry_attempts = 2, -- Reintentos
    @retry_interval = 10; -- Intervalo en minutos
GO
  
```

6 PROGRAMACIÓN DEL JOB (SCHEDULE MENSUAL)



Primer día
del mes



01:00 AM
(01:00)



SQL Server Agent
Ejecuta el Job



Migración a
Histórico



Eliminación de
registros (>180 días)



Registro en
Histórico



Sistema limpio
y optimizado!



PROYECTOS 4 Y 5

PROYECTO 4

1 ENUNCIADO

Crear un Alert para errores de severidad ≥ 17 que inserte un registro detallado como evidencia en la tabla de registro de errores.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
CREATE ALERT [Alerta_Severidad_17]
FOR ERROR_NUMBER = 50000
WITH SEVERITY = 17
ADD EVENTLOG ENTRY 'Alerta de Error Crítico',
DATABASE MAIL PROFILE PerfilDBA,
DESCRIPTION =
'Se generó un error de severidad alta.',
ENABLE;
```



3 JUSTIFICACIÓN TÉCNICA



- Las alertas monitorean eventos críticos de manera automática.
- Severidad ≥ 17 indica problemas que afectan la operación del servidor.
- Se registra en el log y se envía notificación inmediata al DBA.

4 BUENAS PRÁCTICAS



- Alertas configuradas solo para errores críticos.
- Notificaciones claras y oportunas.
- Registro en tabla de auditoría para trazabilidad.
- Revisar y ajustar umbrales de severidad.

PROYECTO 5

1 ENUNCIADO

Implementar un informe automático en el sistema que se ejecute cada 30 días y envíe un correo con el reporte de trámites registrados en los últimos 30 días.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
-- Job que genera el reporte cada 30 días
EXEC msdb.dbo.sp_add_job
@job_name = 'Reporte_30_Dias';

EXEC msdb.dbo.sp_add_schedule
@schedule_name = 'Cada_30_Dias',
@freq_type = 4, -- mensual
@freq_interval = 1,
@freq_relative_interval = 1,
@freq_recurrence_factor = 1,
@active_start_date = 20240501;

EXEC msdb.dbo.sp_attach_schedule
@job_name = 'Reporte_30_Dias',
@schedule_name = 'Cada_30_Dias';

EXEC msdb.dbo.sp_add_jobstep
@job_name = 'Reporte_30_Dias',
@step_name = 'Generar_Reporte_',
@subsystem = 'TSQL!',
@command = 'EXEC dbo.usp_ReporteTramites @Dias = 30';

EXEC msdb.dbo.sp_add_jobserver
@job_name = 'Reporte_30_Dias!';
```



3 JUSTIFICACIÓN TÉCNICA



- Automatiza la generación y envío del reporte periódico.
- Reduce intervención manual y errores humanos.
- Mantiene informados a los responsables con datos actualizados.
- Mejora la trazabilidad y control de trámites.

4 BUENAS PRÁCTICAS



- Automatización de procesos repetitivos.
- Uso de SQL Server Agent para programación.
- Reportes claros y bien estructurados.
- Validar el envío de correos periódicamente.
- Respaldo reportes generados.



TEMA 2

USO DE DATABASE MAINTENANCE PLANS



Mantén tu base de datos en óptimas condiciones como un verdadero héroe.



PROYECTO 1: PLAN DE MANTENIMIENTO DIARIO

1 ENUNCIADO

Crear un Plan de mantenimiento que realice diariamente las tareas completas de la base de datos (CheckDB, Index).



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
BACKUP DATABASE @databaseName  
TO DISK = 'C:\Backups\@databaseName.bak'  
WITH COMPRESSION, CHECKSUM;  
  
DBCC CHECKDB(@databaseName)  
WITH NO_INFOMSGS;  
  
ALTER INDEX ALL ON @databaseName REBUILD;  
UPDATE STATISTICS @databaseName;
```

3 JUSTIFICACIÓN TÉCNICA

- ✓ Respaldo completo protege toda la información.
- ✓ Usar COMPRESIÓN reduce espacio y tiempo.
- ✓ CHECKDB valida la integridad del backup.
- ✓ REBUILD y UPDATE garantizan rendimiento.

4 BUENAS PRÁCTICAS

- 🥇 Programar en horario de baja actividad.
- 🥈 Almacenar backups en ruta segura.
- 🥉 Verificar periódicamente los trabajos.
- 🏆 Registrar la ejecución en SQL Server Agent.

PROYECTO 2: REORGANIZACIÓN SEMANAL DE ÍNDICES

1 ENUNCIADO

Crear un plan semanal para reorganizar índices en las tablas: Persona, Tramite, TrabajoInvestigacion y Sustentacion.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
ALTER INDEX ALL ON Persona REORGANIZE;  
ALTER INDEX ALL ON Tramite REORGANIZE;  
ALTER INDEX ALL ON TrabajoInvestigacion REORGANIZE;  
ALTER INDEX ALL ON Sustentacion REORGANIZE;
```

3 JUSTIFICACIÓN TÉCNICA

- ✓ Reorganizar índices reduce fragmentación leve.
- ✓ Mejora el rendimiento de las consultas.
- ✓ Evita bloqueos largos (menos invasivo que REBUILD).

4 BUENAS PRÁCTICAS

- 🥇 Ejecutar semanalmente.
- 🥈 Monitorear fragmentación antes de ejecutar.
- 🥉 Solo en tablas críticas del sistema.
- 🏆 Registrar la ejecución en tabla de auditoría.

ELEMENTOS CLAVE



BACKUP

Protege tu información.



CHECKDB

Verifica la integridad.



**REBUILD
/REORGANIZE**
Optimiza los índices.



STATISTICS

Actualiza estadísticas para mejores planes de ejecución.



SQL SERVER AGENT

Automatiza y programa tareas confiables.



OBJETIVO: Mantener la base de datos siempre en óptimas condiciones, asegurando rendimiento, integridad y disponibilidad.



TEMA 2

USO DE DATABASE MAINTENANCE PLANS



MANTÉN TU BASE DE DATOS EN ÓPTIMAS CONDICIONES COMO UN VERDADERO HÉROE.

PROYECTO 3: OPTIMIZACIÓN DE ÍNDICES Y ESTADÍSTICAS

1 ENUNCIADO

Diseñar un plan que:

- Reorganice índices < 30 %
- Reconstruya índices > 30 %
- Actualice estadísticas.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
DECLARE @frag FLOAT;
SELECT @frag = avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL,
NULL, 'SAMPLED')
WHERE object_id = OBJECT_ID('Persona');

IF @frag < 30
    ALTER INDEX ALL ON Persona REORGANIZE;
ELSE
    ALTER INDEX ALL ON Persona REBUILD;

UPDATE STATISTICS Persona;
```

3 JUSTIFICACIÓN TÉCNICA

- ✓ Maneja inteligentemente la fragmentación.
- ✓ REORGANIZE optimiza fragmentación baja.
- ✓ REBUILD resuelve fragmentación alta.
- ✓ UPDATE STATISTICS mantiene el plan de ejecución óptimo.

4 BUENAS PRÁCTICAS

- 1 Usar sys.dm_db_index_physical_stats para medir.
- 1 Ejecutar en horarios de mantenimiento.
- 1 Aplicar solo a tablas necesarias.
- 1 Registrar resultados para seguimiento.

PROYECTO 4: MANTENIMIENTO COMPLETO AUTOMATIZADO

1 ENUNCIADO

Automatizar el mantenimiento completo de la base de datos incluyendo:

Backup, CHECKDB, limpieza del historial, reorganización de índices y actualización de estadísticas.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backup\GradosTitulos.bak'
WITH COMPRESSION, CHECKSUM;

DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;

DELETE FROM LogHistorico
WHERE Fecha < DATEADD(DAY, -180, GETDATE());

ALTER INDEX ALL ON Persona REORGANIZE;
UPDATE STATISTICS GradosTitulos;
```

3 JUSTIFICACIÓN TÉCNICA

- ✓ Integra todas las tareas de mantenimiento.
- ✓ CHECKDB detecta errores de integridad.
- ✓ Limpieza del historial libera espacio.
- ✓ Mejora el rendimiento general del sistema.

4 BUENAS PRÁCTICAS

- 1 Programar con Maintenance Plans.
- 1 Ejecutar en ventana de mantenimiento.
- 1 Notificar resultados (SQL Server Agent).
- 1 Usar compresión y validaciones.

PROYECTO 5: PLAN INTELIGENTE POR FRAGMENTACIÓN

1 ENUNCIADO

Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.



2 SCRIPT DE LA SOLUCIÓN (T-SQL)

```
DECLARE @frag FLOAT;
SELECT @frag = avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(),
NULL, NULL, NULL, 'SAMPLED')
WHERE object_id = OBJECT_ID('Tramite');

IF @frag < 30
    ALTER INDEX ALL ON Tramite REORGANIZE;
ELSE
    ALTER INDEX ALL ON Tramite REBUILD;

UPDATE STATISTICS Tramite;
```

3 JUSTIFICACIÓN TÉCNICA

- ✓ Evalúa fragmentación antes de tomar acción.
- ✓ Optimiza recursos del servidor.
- ✓ Mantiene alto rendimiento.
- ✓ Decisión automática y eficiente.

4 BUENAS PRÁCTICAS

- 1 Uso de cursores/DMV.
- 1 Registro en tabla de auditoría.
- 1 Ejecución en horarios óptimos.
- 1 Monitoreo continuo de fragmentación.





TEMA 3

AUTOMATIZACIÓN

CON T-SQL

Y POWERSHELL

¡NIVEL COMPLETADO!

MISIÓN: AUTOMATIZA TUS TAREAS,
RESCATA LOS DATOS Y SALVA EL REINO CHAMPIÑÓN

1 ENUNCIADO

SITUACIÓN ACTUAL (NIVEL DEL PROBLEMA)

La Universidad necesita automatizar el respaldo completo de su base de datos académica (GradosTitulos) para proteger la información crítica ante fallos o pérdidas.

OBJETIVO DEL NIVEL

Crear un procedimiento almacenado que genere un respaldo completo y automático de la base de datos usando T-SQL, ejecutado y programado mediante PowerShell.

REQUERIMIENTOS

- Usar la base de datos real: GradosTitulos.
- El respaldo debe almacenarse en una ruta segura.
- El procedimiento debe ser ejecutable desde PowerShell.
- Generar un archivo .bak con fecha y hora.
- Automatizar su ejecución diaria.

2 SCRIPT T-SQL

```
-- PROCEDIMIENTO ALMACENADO: RESPALDO COMPLETO
USE GradosTitulos;
GO
CREATE OR ALTER PROCEDURE dbo.usp_BackupCompleto
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Database NVARCHAR(128) = 'GradosTitulos';
    DECLARE @FechaHora NVARCHAR(20) =
        CONVERT(NVARCHAR(20), GETDATE(), 112) + '_' +
        REPLACE(CONVERT(NVARCHAR(8), GETDATE(), 108), ':', '');
    DECLARE @Ruta NVARCHAR(260) =
        'D:\Backups\GradosTitulos_' + @FechaHora + '.bak';

    DECLARE @SQL NVARCHAR(MAX) =
        'BACKUP DATABASE [' + @Database + ']'
        'TO DISK = N''' + @Ruta + '''
        WITH INIT, COMPRESSION, STATS = 10;';

    EXEC sp_executesql @SQL;

    PRINT '✅ Respaldo completo generado correctamente.';
    PRINT '✅ Archivo generado: ' + @Ruta;
END
GO
```

EJECUCIÓN CON POWERSHELL

```
# Configuración
$server = "localhost"
$database = "GradosTitulos"
$query = "EXEC dbo.usp_BackupCompleto"

# Conexión y ejecución
$conn = New-Object System.Data.SqlClient.SqlConnection
$conn.ConnectionString = "Server=$server;Database=$database;
Integrated Security=True;"
$cmd = $conn.CreateCommand()
$cmd.CommandText = $query
$conn.Open()
$cmd.ExecuteNonQuery()
$conn.Close()

Write-Host "✅ Respaldo ejecutado desde PowerShell correctamente."
```

3 JUSTIFICACIÓN TÉCNICA

- Automatización confiable:** El procedimiento almacenado centraliza la lógica de respaldo en el servidor SQL, reduciendo errores humanos.
- Trazabilidad:** El nombre del archivo incluye fecha y hora exacta, facilitando la identificación y gestión de respaldos.
- Rendimiento optimizado:** Se usa COMPRESSION para reducir el tamaño del archivo .bak, ahorrando espacio en disco.
- Seguridad:** El respaldo se almacena en una ruta segura en el servidor, evitando pérdida de datos críticos.
- Integración T-SQL + PowerShell:** Permite ejecutar el respaldo de forma remota y programada (ej. con Task Scheduler).
- Escalable:** Puede adaptarse para respaldos diferenciales, transaccionales o múltiples bases de datos.

4 BUENAS PRÁCTICAS DEL DBA

- | | | | | | |
|---|---|--|---|--|---|
| USAR SET NOCOUNT ON
Evita mensajes innecesarios, mejora el rendimiento del procedimiento. | USAR COMPRESION
Reduce el tamaño del archivo .bak, ahorrando espacio y tiempo de transferencia. | NOMBRES DINÁMICOS
Incluir fecha y hora en el nombre del archivo evita sobrescrituras accidentales. | AUTOMATIZACIÓN PROGRAMADA
Ejecutar el respaldo diario con PowerShell + Task Scheduler garantiza consistencia. | VERIFICAR RESPALDOS
Probar la restauración periódicamente para asegurar la integridad. | RUTAS SEGURAS
Almacenar en discos exclusivos o servidores dedicados para respaldos. |
|---|---|--|---|--|---|

¡MISIÓN CUMPLIDA!

Has automatizado el respaldo de GradosTitulos y protegido el Reino Champiñón.

TEMA 3 ★

AUTOMATIZACIÓN CONT-SQL Y POWERSHELL

¡NIVEL AVANZADO COMPLETADO!

PROYECTO 3

Generación de archivo Excel y copia de respaldo

1. ENUNCIADO

Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel y almacene una copia en una carpeta de respaldo.



2. SCRIPT DE LA SOLUCIÓN

T-SQL (Procedimiento)

```
USE dbw_my_ContenTransitoSiria
GO
CREATE OR ALTER PROCEDURE
dbo.usp_ContenTransitoDiario
AS
BEGIN
    SET NOCOUNT ON;
    SELECT CAST(GETDATE() AS DATE) AS Fecha,
           COUNT(*) AS TotalTramites
    FROM dw_my_ContenTransitoSiria
    WHERE CAST(fechaTramite AS DATE)
           = CAST(GETDATE() AS DATE)
    FROM Trámite.Trámite
    WHERE fechaRegistro = CAST(GETDATE() AS DATE);
    AND FechaRegistro = CAST(GETDATE() AS DATE);
END;
```

PowerShell (ImportExcel)

```
# Requiere: Install-Module ImportExcel
$server = "localhost"
$db = "ContenTransito"
$fecha = Get-Date -Format "yyyyMMdd"
$outDir = "C:\Reportes\ContenTransito_$fecha.xlsx"
$backup = "C:\Backups\ContenTransito\$fecha.xlsx"
$sql = "EXEC dbo.usp_ContenTransitoDiario"

$data = Invoke-Sqlcmd -ServerInstance $server -
Database $db -Query $sql
$data | Export-Excel -Path $outDir -AutoSize -
BoldTopRow -WorksheetName "Resumen Diario"

Copy-Item -Path $outDir -Destination $backup -Force

Write-Host "Excel generado: $outDir" -
ForegroundColor Green
Write-Host "Copia de respaldo: $backup" -
ForegroundColor Cyan
```

3. JUSTIFICACIÓN TÉCNICA

- ★ El procedimiento centraliza la consulta diaria de trámites garantizando consistencia.
- ★ ImportExcel crea archivos .xlsx listos para análisis en Excel.
- ★ La copia de respaldo asegura disponibilidad del archivo ante fallos.
- ★ Automatización ideal para reportes ejecutivos diarios.

4. BUENAS PRÁCTICAS

- ✓ AutoSize y BoldTopRow para mejor legibilidad.
- ✓ Separar carpetas: reportes diarios y respaldos.
- ✓ Validar existencia del módulo ImportExcel.
- ✓ Manejo de errores con PowerShell.

PROYECTO 4

Detección de docentes inactivos y generación de reporte

1. ENUNCIADO

Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo con el reporte (CSV).

2. SCRIPT (VISTA)

T-SQL (Vista)

```
USE dbw_DocenteInactivos
GO
CREATE OR ALTER VIEW
dbo.vw_DocentesInactivos
AS
SELECT d.DocenteID, d.Apellidos,
       d.Nombres, d.Email,
       MAX(a.FechaActividad) AS
       UltimaActividad
FROM Academico.Docente d
LEFT JOIN Academico.ActividadDocente a
ON d.DocenteID = a.DocenteID
GROUP BY d.DocenteID, d.Apellidos,
         d.Nombres, d.Email
HAVING MAX(a.FechaActividad) <
DATEADD(YEAR, -1, GETDATE());
```

PowerShell

```
$server = "localhost"
$db = "DocentesDB"
$out = "C:\Reportes\DocentesInactivos_$(Get-Date -Format 'yyyyMMdd').csv"

$sql = "SELECT * FROM
dbo.vw_DocentesInactivos"

$data = Invoke-Sqlcmd -ServerInstance
$server -Database $db -Query $sql

$data | Export-Csv -Path $out -NoTypeInformation
-Encoding UTF8

Write-Host "Reporte generado: $out" -
ForegroundColor Green
```

3. JUSTIFICACIÓN TÉCNICA

- ★ La vista identifica docentes con más de 365 días sin actividad.
- ★ PowerShell automatiza la exportación a CSV.
- ★ Se usa CSV por compatibilidad universal.
- ★ Reporte útil para gestión académica y toma de decisiones.

4. BUENAS PRÁCTICAS

- ✓ Manejo de caso sin datos.
- ✓ Orden descendente para priorizar los más inactivos.
- ✓ Nombre de archivo con fecha.
- ✓ Usar UTF8 para compatibilidad.

PROYECTO 5

Automatización completa con respaldos, CHECKDB, compresión y limpieza

1. ENUNCIADO

Implementar una solución de automatización completa que ejecute respaldos, valide con CHECKDB, genere reportes, comprima logs respaldos (ZIP) y elimine los registros > 30 días.

2. SCRIPT DE LA SOLUCIÓN (COMPLETO)

PowerShell (Completo)

```
$server = "localhost"
$db = "AcademicoDB"
$backupDir = "C:\Backups"
$reportDir = "C:\Reportes"
$fecha = Get-Date -Format "yyyyMMdd_HHmmss"

# 1. Respaldo completo
Backup-SqlDatabase -ServerInstance $server -Database $db -BackupFile
("$backupDir\$db_$fecha.bak") -CompressionOption On

# 2. CHECKDB
Invoke-Sqlcmd -ServerInstance $server -Database $db -Query "DBCC
CHECKDB ('$db') WITH NO_INFOMSGS;" | Out-File "$reportDir\CHECKDB_
$db_$fecha.txt" -Encoding UTF8

# 3. Compresión de respaldo
Compress-Archive -Path "$backupDir\$db_$fecha.bak" -DestinationPath
"$backupDir\$db_$fecha.zip"

# 4. Limpieza: eliminar registros > 30 días
Invoke-Sqlcmd -ServerInstance $server -Database $db -Query
"DELETE FROM dbo.Logs WHERE FechaLog < DATEADD(DAY, -30, GETDATE())"

Write-Host "Proceso completado correctamente." -ForegroundColor Green
```

3. JUSTIFICACIÓN TÉCNICA

- ★ El respaldo asegura recuperación ante fallos.
- ★ CHECKDB valida integridad de la base de datos.
- ★ Compresión reduce espacio en disco y costos de almacenamiento.
- ★ Limpieza mantiene la base optimizada y rápida.

4. BUENAS PRÁCTICAS

- ✓ Registro de todas las acciones.
- ✓ Manejo de errores robusto.
- ✓ Compresión para ahorrar espacio.
- ✓ Eliminación automática de datos antiguos > 30 días.
- ✓ Separación de carpetas: backups y logs.

¡MISIÓN CUMPLIDA!

Automatiza tus procesos y protege tu base de datos como un verdadero héroe del Reino Champiñón.



TEMA 4



MONITOREO PROACTIVO

MARIO PROTEGE TU BASE DE DATOS!

PROYECTO 1

Envío de correo con resumen de trámites pendientes



1. ENUNCIADO

Crear un procedimiento almacenado que genere un resumen diario de los trámites pendientes y lo envíe por correo electrónico usando CDO.Y (Database Mail) y sp_send_dbmail.



2. SCRIPT T-SQL

```
CREATE OR ALTER PROCEDURE dbo.usp_EnviarResumenTramitesPendientes
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @TotalPendientes INT, @FechaCorte NVARCHAR(10),
            @Resumen NVARCHAR(MAX), @Asunto NVARCHAR(200);

    SELECT @TotalPendientes = COUNT(*)
    FROM dbo.Tramite
    WHERE EstadoTramiteID IN ('Pendiente', 'En Proceso');

    SELECT @FechaCorte = CONVERT(NVARCHAR(10), GETDATE(), 120);
    SET @Resumen = 'Resumen de Trámites Pendientes - ' + @FechaCorte
    + CHAR(13) + CHAR(10) + 'Total Pendientes: '
    + CAST(@TotalPendientes AS NVARCHAR) + CHAR(13) + CHAR(10)
    + 'Detalle por Estado: ' + CHAR(13) + CHAR(10)
    + (SELECT Estado = E.Nombre, Total = COUNT(*)
    FROM dbo.Tramite T
    INNER JOIN dbo.EstadoTramite E
    ON T.EstadoTramiteID = E.EstadoTramiteID
    WHERE E.Nombre IN ('Pendiente', 'En Proceso')
    GROUP BY E.Nombre
    FOR XML PATH(''), TYPE).value('.', 'NVARCHAR(MAX)');

    SET @Asunto = 'Resumen Diario de Trámites Pendientes - ' + @FechaCorte;

    EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'PerfilTramites',
    @recipients = 'administracion@upla.edu.pe',
    @subject = @Asunto,
    @body = @Resumen,
    @body_format = 'TEXT';

END
GO
```



3. JUSTIFICACIÓN TÉCNICA

- Automatiza el envío diario del resumen de trámites pendientes sin intervención humana.
- Permite un control oportuno y centralizado de la carga de trabajo.
- Utiliza Database Mail para un envío seguro y confiable.

4. BUENAS PRÁCTICAS

- Uso de sp_send_dbmail para evitar configuraciones externas.
- Asunto y cuerpo dinámicos para mejor trazabilidad.
- Control de errores y validaciones implícitas en el procedimiento.

PROYECTO 2

Auditoría de errores en SP_RegistrarTramite



1. ENUNCIADO

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento Tramite.SP_RegistrarTramite, utilizando TRY...CATCH, ERROR_MESSAGE, ERROR_NUMBER y ERROR_LINE.



2. SCRIPT T-SQL

```
CREATE OR ALTER PROCEDURE Tramite.SP_RegistrarTramite
@TramiteID INT, @UsuarioID INT, @Observaciones NVARCHAR(1000) = NULL,
@Resultado INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRAN
            INSERT INTO TramiteHistorial (TramiteID, EstadoTramiteID,
            FechaRegistro, UsuarioID, Observaciones, EstadoObservacionID)
            VALUES (@TramiteID, 1, GETDATE(), @UsuarioID, @Observaciones, 1);

            INSERT INTO Tramite (TramiteID, TramiteTipoID, FechaRegistro,
            Observaciones, EstadoTramiteID)
            VALUES (@TramiteID, 1, GETDATE(), @Observaciones, 1);

        COMMIT TRAN
        SET @Resultado = 1;
    END TRY
    BEGIN CATCH
        DECLARE @ErrNumber INT = ERROR_NUMBER(),
                @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE(),
                @ErrLine INT = ERROR_LINE(), @ErrProc NVARCHAR(200) = ERROR_PROCEDURE();

        INSERT INTO TramiteAuditError (
            FechaError, Procedimiento, NumeroError, MensajeError, LineaError, UsuarioID)
        VALUES (GETDATE(), 'SP_RegistrarTramite', @ErrNumber, @ErrMsg, @ErrLine, @ErrProc);

        ROLLBACK TRAN
        SET @Resultado = -1;
    END CATCH
END
```



3. JUSTIFICACIÓN TÉCNICA

- Permite capturar y registrar cualquier error que ocurra durante la ejecución del SP.
- Facilita la trazabilidad y solución de problemas.
- Evita pérdida de información crítica mediante el log de auditoría.

4. BUENAS PRÁCTICAS

- Uso de TRY....CATCH para manejo robusto de errores.
- Registro detallado: número, mensaje, línea y procedimiento.
- No se expone el error al usuario final (manejo interno).

5. SIMULACIÓN DE SOLUCIÓN

```
-- Ejecución del envío de resumen diario
EXEC dbo.usp_EnviarResumenTramitesPendientes;

-- Ejecución correcta de SP_RegistrarTramite
DECLARE @Resultado INT;
EXEC Tramite.SP_RegistrarTramite
@TramiteID = 1001,
@UsuarioID = 25,
@Observaciones = 'Nuevo trámite ingresado',
@Resultado = @Resultado OUTPUT;
SELECT @Resultado AS Resultado;

-- Ejecución con error (ej. TramiteID duplicado)
EXEC Tramite.SP_RegistrarTramite
@TramiteID = 1001,
@UsuarioID = 25,
@Observaciones = 'Duplicado intencional',
@Resultado = @Resultado OUTPUT;
SELECT @Resultado AS Resultado;
GO
```



6. SIMULACIÓN DEL RESULTADO

Tabla: TramiteAuditError (Ejemplo)

FechaError	Procedimiento	NumeroError	MensajeError	LineaError	UsuarioID
2025-05-17 10:15:22	SP_RegistrarTramite	2627	Violation of PRIMARY KEY constraint... Cannot insert duplicate key...	25	25
2025-05-17 10:20:45	SP_RegistrarTramite	547	The INSERT statement conflicted with the FOREIGN KEY constraint...	31	25

Tabla: Tramite (Resumen)

TramiteID	TramiteTipoID	FechaRegistro	EstadoTramiteID	Observaciones
1001	1	2025-05-17 10:10:01	1	Nuevo trámite registrado
1002	1	2025-05-17 10:12:30	2	En revisión por administración



TEMA 4

MONITOREO PROACTIVO

¡MARIO PROTEGE TU BASE DE DATOS!

PROYECTO 3

Alerta por crecimiento del log
(80% del tamaño)

1 ENUNCIADO

Monitorear el crecimiento del archivo de log y generar una alerta cuando supere el 80% del tamaño configurado, enviando correo electrónico.



2 SCRIPT T-SQL

```
CREATE OR ALTER PROCEDURE dbo.usp_MonitoreoLog
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @TamLog BIGINT, @UsadoLog BIGINT,
            @PctUsado INT, @DbName SYSNAME;

    SELECT @DbName = DB_NAME();

    SELECT @TamLog = SUM(size) * 8.0 / 1024
    FROM sys.database_files WHERE type_desc = 'LOG';

    SELECT @UsadoLog = log_reuse_wait_desc = 'NOTHING'
    * (RECENT log_space_used_percent
    FROM sys.dm_db_log_space_usage);

    SET @PctUsado = CAST((@UsadoLog * 100.0) / @TamLog AS INT);

    IF @PctUsado >= 80
    BEGIN
        EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'PerfilAlertasDB',
        @recipients = 'dba@empresa.com',
        @subject = 'ALERTA: Log al 80%',
        @body = CONCAT('Log actual: ', @PctUsado, '%'),
        @body_format = 'TEXT';
    END
```

3 JUSTIFICACIÓN TÉCNICA

- Previene crecimiento descontrolado del log.
- Evita que la BD entre en estado de emergencia.
- Permite acción temprana para liberar espacio y mantener la continuidad del servicio.

4 BUENAS PRÁCTICAS

- Uso de sp_send_dbmail para alertas.
- Verificación del % usado para umbral crítico.
- Consulta a vistas dinámicas de SQL Server.
- Fácil programación en SQL Agent.

5 SIMULACIÓN DEL RESULTADO

ALERTA: Log de GradosTitulos
al 80%Uso del log 81%

PROYECTO 4

Monitoreo de tiempo de ejecución
de procedimientos y ranking

1 ENUNCIADO

Registrar automáticamente el tiempo de ejecución de los procedimientos ejecutados y generar un ranking de los que consumen más tiempo para optimizarlos.



2 SCRIPT T-SQL (REGISTRO)

```
CREATE TABLE dbo.LogEjecucionProc (
    IdLog INT IDENTITY PRIMARY KEY,
    FechaEjecucion DATETIME DEFAULT GETDATE(),
    NombreProc SYSNAME,
    DuracionMs INT,
    UsuarioEjecutor SYSNAME,
    Estado BIT,
    Mensaje NVARCHAR(4000)
);

INSERT INTO dbo.LogEjecucionProc
(NombreProc, DuracionMs, UsuarioEjecutor, Estado)
VALUES (@Proc, @DuracionMs, SYSUSER_NAME(), 1);
```

3 SCRIPT T-SQL (RANKING)

```
SELECT TOP 10
    NombreProc,
    COUNT(*) AS Ejecuciones,
    AVG(DuracionMs) AS PromedioMs,
    MAX(DuracionMs) AS MaximoMs,
    SUM(DuracionMs) AS TotalMs
FROM dbo.LogEjecucionProc
WHERE FechaEjecucion >= DATEADD(DAY, -1, GETDATE())
GROUP BY NombreProc
ORDER BY TotalMs DESC;
```

4 JUSTIFICACIÓN TÉCNICA

- Permite identificar procedimientos que consumen más tiempo.
- Facilita la optimización basada en datos reales de ejecución.
- Mejora el rendimiento general y reduce la carga en el servidor.

5 BUENAS PRÁCTICAS

- Registro consistente y centralizado.
- Uso de DATETIME y milisegundos para precisión.
- Ranking por tiempo total para priorizar optimización.

6 SIMULACIÓN DEL RESULTADO (RANKING)

NombreProc	Ejecuciones	PromedioMs	MaximoMs	TotalMs
usp_GenerarReporte	124	12500	22000	1530000
usp_CargarVentas	98	9500	18000	931000
usp_ValidarMetricas	76	6700	15000	509200
usp_ActualizarDatos	64	4200	9000	240800
usp_EnejarCorreos	32	3100	7000	99300

PROYECTO 5

Sistema integral de monitoreo (bloqueos, consultas lentas, CPU alto, alertas)

1 ENUNCIADO

Sistema integral de monitoreo para capturar bloqueos, consultas lentas, uso alto de CPU y generar alertas. Toda la información se registra para análisis y acción oportuna.



2 SCRIPT DE MONITOREO

2.1 Captura de bloqueos

```
INSERT INTO dbo.Bloqueos
SELECT GETDATE(), request_session_id,
resource_type, resource_description,
request_mode, request_status
FROM sys.dm_tran_locks
WHERE request_status = 'WAIT';
```



2.4 Estado de espera (bloqueos)

```
INSERT INTO dbo.EsperaBloqueos
SELECT GETDATE(), session_id, wait_type,
wait_time_ms, blocking_session_id
FROM sys.dm_exec_requests
WHERE blocking_session_id <> 0;
```



2.2 Consultas lentas (CPU)

```
INSERT INTO dbo.ConsultasLentas
SELECT GETDATE(), text, total_worker_time,
execution_count, total_elapsed_time
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle)
WHERE total_worker_time/1000 > 5000;
```



2.5 Alertas automáticas

```
IF EXISTS (SELECT 1 FROM dbo.CargaCPU
WHERE FechaRegistro >= DATEADD(MINUTE, -5, GETDATE())
AND CpuCPU > 80)
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'PerfilAlertasDB',
    @recipients = 'dba@empresa.com',
    @subject = 'ALERTA: CPU Alto',
    @body = 'Uso de CPU superior al 80%';
END
```



2.3 Sesiones con alto CPU (>80%)

```
INSERT INTO dbo.CargaCPU
SELECT GETDATE(), session_id, cpu_time,
cpu_time/MAX(cpu_time) AS PctCargaCPU
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
AND cpu_time / (total_scheduled_time/100.0) > 80;
```



2.6 Limpieza de datos antiguos

```
DELETE FROM dbo.MonitorioHistorico
WHERE FechaRegistro < DATEADD(DAY, -90, GETDATE());
```



3 JUSTIFICACIÓN TÉCNICA

- Visión 360° del rendimiento de la base de datos.
- Permite detectar y resolver problemas antes que afecten a los usuarios.
- Historial para análisis de tendencias y capacidad.

4 BUENAS PRÁCTICAS

- Registro estructurado y centralizado.
- Umbral configurable para alertas.
- Limpieza periódica de datos.
- Uso de vistas dinámicas de SQL Server.
- Envío de alertas con sp_send_dbmail.

5 DASHBOARD DE ESTADO (RESUMEN)

6 UMBRALES CONFIGURADOS

7 HISTÓRICO DE ALERTAS (ÚLTIMAS)

Bloqueos Activos	Consultas Lentas	CPU Alto (>80%)	Alertas Enviadas
3	7	5	1

Métrica	Umbral	Acción
CPU Uso	> 80%	Enviar alerta
Consulta Lenta	> 5 seg	Registrar y alertar
Bloqueo	> 30 seg	Registrar y alertar
Espacio Log	> 80%	Enviar alerta

Fecha/Hora	Alerta	Detalle
17/05/2025 10:15	CPU Alto	Sesión 57 - CPU 92%
17/05/2025 10:12	Consulta Lenta	Duración 12.4 seg
17/05/2025 10:08	Bloqueo Detectado	Sesión 23 bloqueada
17/05/2025 10:05	Log al 80%	Log usado 81%